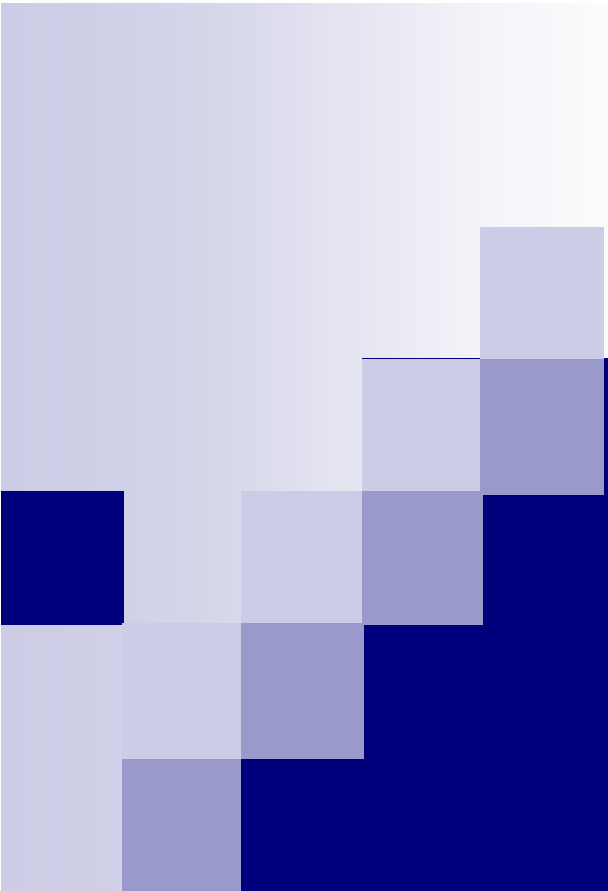


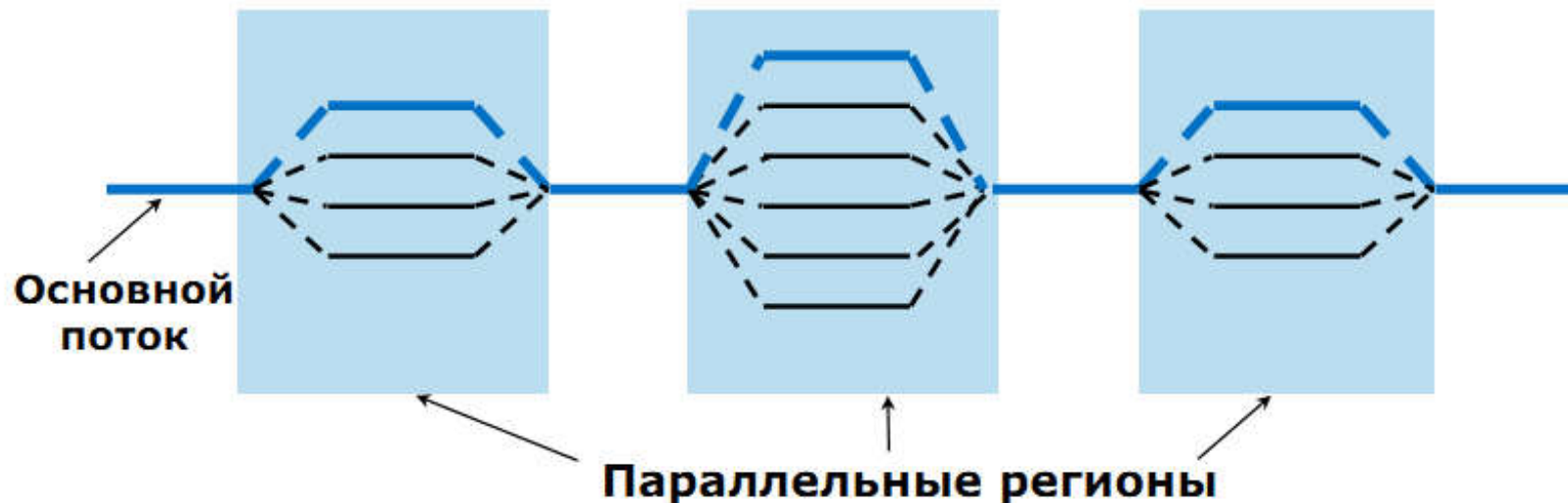


OpenMP

программирование для
систем с общей памятью

Модель

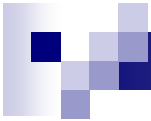
- OpenMP (Open Multi-Processing) – API, предназначенное для программирования многопоточных приложений для систем с общей памятью (для языков C, C++, Fortran)
 - основной поток создает потоки по мере необходимости
 - один код как для последовательной, так и для параллельной версий





Как пользоваться?

- OpenMP – набор параметров директивы `#pragma`
- Если OpenMP поддерживается, то определен макрос `_OPENMP`
- При компиляции используем флаги компиляции
`-fopenmp` (gcc), `-openmp` (Visual Studio)
- Все директивы начинаются с `#pragma omp`
- Каждая директива может иметь несколько дополнительных атрибутов
- Все функции начинаются с `omp_`
- `#include <omp.h>`



#pragma omp parallel

- Создаёт группу потоков (определяется во время выполнения, если не указано иное)

- Код области дублируется между потоками

```
#pragma omp parallel
```

```
{
```

```
    // Код внутри блока выполняется  
параллельно.
```

```
    std::cout << "Hello!" << endl;
```

```
}
```

- В конце области обеспечивается синхронизация потоков и все потоки завершаются



Параметры parallel

- `num_threads` (число)
- `if` (условие)
- `private` (список_локальных_переменных)
- `firstprivate` (список_локальных_переменных)
- `shared` (список_разделяемых_переменных)
- `copyin` (список)
- `reduction` (оператор: список)



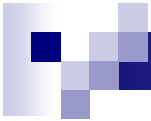
Пример

```
#include <omp.h>
int main () {
    int nthreads, tid;
    // Создание параллельной области
    #pragma omp parallel private(tid)
    {
        // печать номера потока
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);
        // Печать количества потоков - только master
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }
    } // Завершение параллельной области
}
```



#pragma omp for

- Распределяет итерации цикла между потоками
`#pragma omp for`
`for (...) ...`
- Параметры `for`
 - ☐ `private(список)`
 - ☐ `firstprivate(список)`
 - ☐ `lastprivate(список)`
 - ☐ `reduction(оператор: список)`
 - ☐ `ordered`
 - ☐ `schedule(kind[, chunk_size])`
 - ☐ `nowait`



Параметры `schedule`

- **`static`** – итерации делятся на блоки по `chunk` итераций и статически разделяются между потоками; если параметр `chunk` не определен, итерации делятся между потоками равномерно и непрерывно
- **`dynamic`** – распределение итерационных блоков осуществляется динамически (по умолчанию `chunk=1`)
- **`guided`** – размер итерационного блока уменьшается экспоненциально при каждом распределении; `chunk` определяет минимальный размер блока (по умолчанию `chunk=1`)
- **`runtime`** – правило распределения определяется переменной `OMP_SCHEDULE` (при использовании `runtime` параметр `chunk` задаваться не должен)



Пример

```
#include <omp.h>
#define CHUNK 100
#define NMAX 1000
main () {
    int i, n, chunk;
    float a[NMAX], b[NMAX], c[NMAX];
    for (i=0; i < NMAX; i++)
        a[i] = b[i] = i * 1.0;
    n = NMAX; chunk = CHUNK;
    #pragma omp parallel shared(a,b,c,n,chunk)
    private(i)
    {
        #pragma omp for schedule(static,chunk) nowait
        for (i=0; i < n; i++)
            c[i] = a[i] + b[i];
    } // end of parallel section
}
```



#pragma omp sections

- Распределяет задачи между потоками

```
#pragma omp sections
{
    #pragma omp section
    операторы
    #pragma omp section
    операторы
}
```

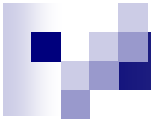
- Параметры for

- ☐ private(список)
- ☐ firstprivate(список)
- ☐ lastprivate(список)
- ☐ reduction(оператор: список)
- ☐ nowait



Пример

```
#include <omp.h>
#define NMAX 1000
main () {
    int i, n;
    float a[NMAX], b[NMAX], c[NMAX];
    for (i=0; i < NMAX; i++)
        a[i] = b[i] = i * 1.0;
    n = NMAX;
    #pragma omp parallel shared(a,b,c,n) private(i)
    {
        #pragma omp sections nowait
        {
            #pragma omp section
                for (i=0; i < n/2; i++)
                    c[i] = a[i] + b[i];
            #pragma omp section
                for (i=n/2; i < n; i++)
                    c[i] = a[i] + b[i];
        } // end of sections
    } // end of parallel section
}
```



Операция редукции

- Параметр **reduction** определяет список переменных, для которых выполняется операция редукции
 - перед выполнением параллельной области для каждого потока создаются копии этих переменных,
 - потоки формируют значения в своих локальных переменных
 - при завершении параллельной области на всеми локальными значениями выполняются необходимые операции редукции, результаты которых запоминаются в исходных (глобальных) переменных
- `reduction (оператор: список_переменных)`



Пример

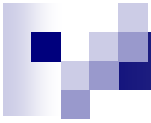
```
#include <omp.h>
main () { // vector dot product
    int i, n, chunk;
    float a[100], result;
    n = 100; chunk = 10;
    result = 0.0;
    for (i=0; i < n; i++)
        a[i] = i * 1.0;

    #pragma omp parallel for default(shared) \
    schedule(static,chunk) reduction(+:result)
    for (i=0; i < n; i++)
        result = result + a[i];
    printf("Final result= %f\n",result);
}
```



Операция редукции

- Переменная, по которой проводится редукция должна быть скалярной
- Переменная в выражении может присутствовать только один раз
- Допустимые операции
 - $+$, $-$, $*$, $/$, $\&$, $^$, $|$, $\&\&$, $||$



Другие директивы

- `#pragma omp master` – определяет фрагмент кода, который должен быть выполнен только основным потоком;
- `#pragma omp single [clause ...]` – определяет фрагмент кода, который должен быть выполнен только одним потоком (любым)
 - `private` (список)
 - `firstprivate` (список)
 - `copyprivate` (список)
 - `nowait`
- `#pragma omp critical [name]` – определяет фрагмент кода, который должен выполняться только одним потоком в каждый текущий момент времени (критическая секция)



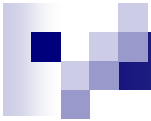
Пример

```
#include <omp.h>
main() {
    int x;
    x = 0;
    #pragma omp parallel shared(x)
    {
        #pragma omp critical
        x = x + 1;
    } // end of parallel section
}
```



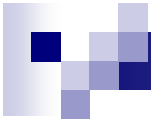

Другие директивы

- `#pragma omp barrier` – определяет точку синхронизации, которую должны достигнуть все потоки для продолжения вычислений
- `#pragma omp atomic` - определяет переменную, доступ к которой (чтение/запись) должен быть выполнен как неделимая операция
 - `+, -, *, /, &, ^, |, >>, <<`



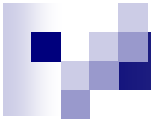
Библиотека функций omp

- `void omp_set_num_threads(int num_threads)` – позволяет назначить максимальное число потоков для использования в следующей параллельной области (если это число разрешено менять динамически). Вызывается из последовательной области программы
- `int omp_get_max_threads(void)` – возвращает максимальное число потоков
- `int omp_get_num_threads(void)` – возвращает фактическое число потоков в параллельной области программы
- `int omp_get_thread_num(void)` - возвращает номер потока
- `int omp_get_num_procs(void)` – возвращает число процессоров, доступных приложению
- `int omp_in_parallel(void)` - возвращает true, если вызвана из параллельной области программы



Библиотека функций omp

- В качестве замков используются общие переменные типа `omp_lock_t`. Данные переменные должны использоваться только как параметры примитивов синхронизации.
- `void omp_init_lock(omp_lock_t *lock)`
- инициализирует замок, связанный с переменной `lock`
- `void omp_destroy_lock(omp_lock_t *lock)` - удаляет замок, связанный с переменной `lock`
- `void omp_set_lock(omp_lock_t *lock)` - заставляет вызвавший поток дожидаться освобождения замка, а затем захватывает его
- `void omp_unset_lock(omp_lock_t *lock)` - освобождает замок, если он был захвачен потоком ранее
- `int omp_test_lock(omp_lock_t *lock)` - пробует захватить указанный замок. Если это невозможно, возвращает `false`



Переменные окружения

- **OMP_SCHEDULE** – определяет способ распределения итераций в цикле, если в директиве for использована клауза `schedule(runtime)`
- **OMP_NUM_THREADS** – определяет число нитей для исполнения параллельных областей приложения
- **OMP_DYNAMIC** – разрешает или запрещает динамическое изменение числа нитей
- **OMP_NESTED** – разрешает или запрещает вложенный параллелизм
- Компилятор с поддержкой OpenMP определяет макрос **_OPENMP**, который может использоваться для условной компиляции отдельных блоков, характерных для параллельной версии программы